

These problems are certainly not insurmountable, but they do show that just introducing threads into an existing system without a fairly substantial system redesign is not going to work at all. The semantics of system calls may have to be redefined and libraries rewritten, at the very least. And all of these things must be done in such a way as to remain backward compatible with existing programs for the limiting case of a process with only one thread. For additional information about threads, see Cook (2008) and Rodrigues et al. (2010).

2.3 EVENT-DRIVEN SERVERS

In the previous section, we have seen two possible designs for a Web server: a fast multithreaded one and a slow single-threaded one. Suppose that threads are not available or not desirable but the system designers find the performance loss due to single threading, as described so far, unacceptable. If nonblocking versions of system calls, such as `read`, are available, a third approach is possible. When a request comes in, the one and only thread examines it. If it can be satisfied from the cache, fine, but if not, a nonblocking disk operation is started.

The server records the state of the current request in a table and then goes and gets the next event. The next event may either be a request for new work or a reply from the disk about a previous operation. If it is new work, that work is started. If it is a reply from the disk, the relevant information is fetched from the table and the reply processed. With nonblocking disk I/O, a reply probably will have to take the form of a signal or interrupt.

In this design, the “sequential process” model that we had in the first two cases is lost. The state of the computation must be explicitly saved and restored in the table every time the server switches from working on one request to another. In effect, we are simulating the threads and their stacks the hard way. A design like this, in which each computation has a saved state, and there exists some set of events that can occur to change the state, is called a **finite-state machine**. This concept is widely used throughout computer science.

In fact, it is very popular in high-throughput servers where even threads are considered too expensive and instead an **event-driven programming paradigm** is used. By implementing the server as a finite state machine that responds to events (e.g., the availability of data on a socket) and interacting with the operating system using non-blocking (or **asynchronous**) system calls, the implementation can be very efficient. Every event leads to a burst of activity, but it never blocks.

Fig. 2-19 shows a pseudo-code example of an event-driven thank-you server (the server thanks every client that sends it a message) that uses the `select` call to monitor multiple network connections (line 17). The `select` determines which file descriptors are ready for receiving or sending data and, looping over them, receives all the messages it can and then tries to send thank-you messages on all corresponding connections that are ready to receive data. In case the server could not

```

0.  /* Preliminaries:
1.  svrSock  : the main server socket, bound to TCP port 12345
2.  toSend   : database to track what data we still have to send to the client
3.             - toSend.put (fd, msg) will register that we need to send msg on fd
4.             - toSend.get (fd) returns the string we need to send msg on fd
5.             - toSend.destroy (fd) removes all information about fd from toSend */
6.
7.  inFds     = { svrSock }           /* file descriptors to watch for incoming data */
8.  outFds     = { }                 /* file descriptors to watch to see if sending is possible */
9.  exceptFds = { }                 /* file descriptors to watch for exception conditions (not used) */
10.
11. char msgBuf [MAX_MSG_SIZE]      /* buffer in which to receive messages */
12. char *thankYouMsg = "Thank you!" /* reply to send back */
13.
14. while (TRUE)
15. {
16.     /* block until some file descriptors are ready to be used */
17.     rdyIns, rdyOuts, rdyExcepts = select (inFds, outFds, exceptFds, NO_TIMEOUT)
18.
19.     for (fd in rdyIns) /* iterate over all the connections that have something for us */
20.     {
21.         if (fd == svrSock)           /* a new connection from a client */
22.         {
23.             newSock = accept (svrSock) /* create new socket for client */
24.             inFds = inFds  $\cup$  { newSock } /* must monitor it also */
25.         }
26.         else
27.         { /* receive the message from the client */
28.             n = receive (fd, msgBuf, MAX_MSG_SIZE)
29.             printf ("Received: %s.\n", msgBuf)
30.
31.             toSend.put (fd, thankYouMsg) /* must still send thankYouMsg on fd */
32.             outFds = outFds  $\cup$  { fd } /* so must monitor this fd */
33.         }
34.     }
35.     for (fd in rdyOuts) /* iterate over all the connections that we can now thank */
36.     {
37.         msg = toSend.get (fd) /* see what we need to send on this connection */
38.         n = send (fd, msg, strlen(msg))
39.         if (n < strlen (thankYouMsg))
40.         {
41.             toSend.put (fd, msg+n) /* remaining characters to send next time */
42.         } else
43.         {
44.             toSend.destroy (fd)
45.             outFds = outFds  $\setminus$  { fd } /* we have thanked this one already */
46.         }
47.     }
47. }

```

Figure 2-19. An event-driven thank-you server (pseudo code).

send the full thank you message, it remembers which bytes it still needs to send, so it can try again later, when there is more space available. We simplified the program to keep it relatively short, by means of pseudo code and not worrying about errors or connections closing. Nevertheless, it illustrates that a single-threaded event-driven server can handle many clients concurrently.

Most popular operating systems offer special, highly optimized event notification interfaces for asynchronous I/O that are much more efficient than `select`. Well-known examples include the `epoll` system call on Linux, and the similar `kqueue` interface on FreeBSD. Windows and Solaris have slightly different solutions. They all allow the server to monitor many network connections at once without blocking on any of them. Because of this Web servers such as `nginx` can comfortably handle ten thousand concurrent connections. This is no trivial feat and even has its own name: the **C10k** problem.

Single-Threaded Versus Multi-Threaded Versus Event-driven Servers

Finally, let us compare the three different ways to build a server. It should now be clear what threads have to offer. They make it possible to retain the idea of sequential processes that make blocking calls (e.g., for disk I/O) and still achieve parallelism. Blocking system calls make programming easier, and parallelism improves performance. The single-threaded server retains the simplicity of blocking system calls but gives up performance.

The third approach, event-driven programming, also achieves high performance through parallelism but uses nonblocking calls and interrupts to do so. It is considered harder to program. These models are summarized in Fig. 2-20.

Model	Characteristics
Threads	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine/event-driven	Parallelism, nonblocking system calls, interrupts

Figure 2-20. Three ways to construct a server.

These three approaches to handle requests from a client apply not only to user programs, but also to the kernel itself where concurrency is just as important for performance. In fact, this is a good moment to point out that this book introduces many operating system concepts with an emphasis on what they mean for user programs, but of course the operating system itself uses these concepts internally also (and some are even more relevant to the operating system than to user programs). Thus, the operating system kernel itself may consist of multithreaded or event-driven software. For instance, the Linux kernel on modern Intel CPUs is a multithreaded operating system kernel. In contrast, MINIX 3 consists of many servers implemented following the model of finite state machine and events.

2.4 SYNCHRONIZATION AND INTERPROCESS COMMUNICATION

Processes frequently need to synchronize and communicate with other processes. For example, in a shell pipeline, the output of the first process must be passed to the second process, and so on down the line. Thus, there is a need for communication between processes, preferably in a well-structured way not using interrupts. In the following sections, we will look at some of the issues related to this **IPC (InterProcess Communication)**.

Very briefly, there are three issues here. The first was alluded to above: how one process can pass information to another. The second has to do with making sure two or more processes or threads do not get in each other's way, for example, two threads in an airline reservation system each trying to grab the last seat on a plane for different customers. The third concerns proper sequencing when dependencies are present: if thread *A* produces data and thread *B* prints them, *B* has to wait until *A* has produced some data before starting to print. We will examine all three of these issues starting in the next section.

It is also important to mention that two of these issues apply to threads as well as to processes with shared memory. The first one—passing information—is clearly easier for threads since they share a common address space by nature. However, the other two—keeping out of each other's hair and proper sequencing—are complicated for threads also. Below we will discuss the problems in the context of processes, but please keep in mind that the same problems and solutions apply to threads.

2.4.1 Race Conditions

In some operating systems, processes that are working together may share some common storage that each one can read and write. The shared storage may be in main memory (possibly in a kernel data structure) or it may be a shared file; the location of the shared memory does not change the nature of the communication or the problems that arise. To see how interprocess communication works in practice, let us now consider a simple but common example: a print spooler. When a process wants to print a file, it enters the file name in a special **spooler directory**. Another process, the **printer daemon**, periodically checks to see if there are any files to be printed, and if there are, it prints them and then removes their names from the directory.

Imagine that our spooler directory has a very large number of slots, numbered 0, 1, 2, ..., each one capable of holding a file name. Also imagine that there are two shared variables, *out*, which points to the next file to be printed, and *in*, which points to the next free slot in the directory. These two variables might well be kept in a two-word file available to all processes. At a certain instant, slots 0–3 are empty (the files have already been printed) and slots 4–6 are full (with the names of files queued for printing). More or less simultaneously, processes *A* and *B* decide they want to queue a file for printing. This situation is shown in Fig. 2-21.